Departamento de Computación
FCEFQyN, Universidad Nacional de Río Cuarto
Asignatura: Análisis y Diseño de Algoritmos I/Diseño de Algoritmos/Algoritmos II
Segundo Cuatrimestre de 2025

Examen Parcial No. 2

Este examen debe ser resuelto en forma individual. No olvide poner su nombre y número de documento en el encabezado de la resolución.

1. Supongamos que se desea viajar en automóvil desde un punto A hasta un punto B a lo largo de una carretera, minimizando la cantidad de veces que se debe detener para cargar combustible.

   Se conoce de antemano la capacidad del tanque de combustible, que permite recorrer como máximo L kilómetros cuando está lleno. Asimismo, se conoce la ubicación de n estaciones de servicio a lo largo del camino, cuyas posiciones son $x_1, x_2, \ldots, x_n$, donde x_i indica la distancia desde el punto A hasta la i-ésima estación ($0 < x_1 < x_2 < \cdots < x_n < B$).

   Diseñe, e implemente en java, un algoritmo *greedy* que determine el número mínimo de veces que es necesario detenerse para cargar combustible con el fin de llegar desde A hasta B. Indique la estrategía greedy utilizada y el tiempo de ejecución de su solución.

2. En bioinformática, es frecuente la necesidad de comparar el ADN de dos o más organismos. Una secuencia de ADN se representa como una cadena de letras que representan cada uno de los nucleótidos posibles: A (adenina), G (guanina), C (citosina) y T (tiamina).

   Por ejemplo: `ACCGGTCGGGATGCACCTGAGAAAGCGG`.

   Un posible criterio de "similitud" entre secuencias de ADN es encontrar la serie compartida más larga de nucleótidos que aparezca en las secuencias de ADN, aún en forma no consecutiva.

   Por ejemplo, para las secuencias:

   $$S_1 = \texttt{AGCGTAG}, \quad S_2 = \texttt{GTCAGA}$$

   la serie compartida más larga es `GCGA` (de longitud 4).

   Escriba un programa que, dadas dos secuencias de ADN, determine la *longitud* de la serie de nucleótidos compartida más larga, en tiempo polinomial utilizando la técnica *programación dinámica*.

3. El siguiente problema consiste en determinar el número de formas en que un número entero positivo puede escribirse como la suma de dos o más enteros positivos.

   Por ejemplo, el número 6 puede expresarse de las siguientes maneras:

   $$\begin{aligned}
   5+1&=6\\
   4+2&=6\\
   4+1+1&=6\\
   3+3&=6\\
   3+2+1&=6\\
   3+1+1+1&=6\\
   2+2+2&=6\\
   2+1+1+2&=6\\
   2+1+1+1+1&=6\\
   1+1+1+1+1+1&=6
   \end{aligned}$$

El siguiente programa recursivo calcula el número de formas en que el número n puede escribirse como la suma de dos o más enteros positivos:

```
public static int contarFormasSuma(int n) {
        return contarParticiones(n, n) - 1;
}

private static int contarParticiones(int n, int m) {
        if (n == 0) return 1;
        if (n < 0) return 0;
        if (m == 0) return 0;

        // contarParticiones(n, m - 1): formas de escribir n sin
        // usar el número m.
        // contarParticiones(n - m, m): formas de escribir n usando
        // al menos un número m.
        return contarParticiones(n, m - 1) + contarParticiones(n - m, m);
}
```

La solución recursiva presentada es poco eficiente. Diseñe e implemente un algoritmo en Java que resuelva el problema utilizando la técnica de *programación dinámica*.

4. Considere el problema de, dado un arreglo a de n números enteros, producir un arreglo m conteniendo en cada posición i la multiplicación de todos los elementos en a excepto $a[i]$. El siguiente algoritmo resuelve este problema:

```
public int[] multiplicaciones(int[] a) {
    int[] m = new int[a.length];
    for (int i = 0; i< m.length; i++) {
        m[i] = mult(a, 0, i-1) * mult(a, i+1, a.length-1);
    }
    return m;
}

private int mult(int[] a, int i, j) {
    if (i>j) return 1;
    return a[i]*mult(a, i+1, j);
}
```

Esta solución sufre de problemas de performance, debido al recómputo de problemas comunes. Identifique cuáles son estos recómputos, y use *memoization* para mejorar la eficiencia del programa.